



CIRES TECHNOLOGIES

Rapport de stage : Etude technique

BENCHMARK DES ARCHITECTURES DE PIPELINE DE MONITORING CLOUD

*Vers une plateforme Cloud Monitoring Overview
augmentée par IA*

Projet : Cloud AI Monitor

Plateforme de supervision Cloud avec Agent IA

Encadré par : EL HANNAOUI IBRAHIM

Farouk Toujgani & Meriche Redouane

Juillet 2026

Table des matières

1	Contexte du projet	3
1.1	Objectif du benchmark	3
2	Les trois patterns d'architecture etudies	4
2.1	Architecture A — Pipeline d'alerting lineaire	4
2.2	Architecture B — Pipeline avec agent de remediation	4
2.3	Architecture C — Pipeline d'observabilite (type Grafana Stack)	5
3	Architecture cible recommandee pour Cloud AI Monitor	7
3.1	Architecture D — Pipeline hybride (Cible)	7
3.2	Principes cles de l'architecture cible	8
3.3	Adequation avec le cahier des charges	8
4	Architecture Cible	9
4.1	Pipeline de traitement de l'Agent IA	9
5	Tableau comparatif	11
6	Taches et fonctionnalites	12
6.1	Authentification securisee	12
6.2	Tableau de bord intelligent	12
6.3	Supervision des machines virtuelles	12
6.4	Supervision des pods	12
6.5	Surveillance des metriques	13
6.6	Collecte automatique des donnees	13
6.7	Detection d'anomalies	13
6.8	Gestion des alertes	13
6.9	Historique	13
6.10	Journalisation	13
6.11	Recherche et filtrage	13
6.12	Gestion des utilisateurs	14
6.13	Gestion des roles	14
6.14	Parametrage du systeme	14
6.15	Visualisation des performances	14
6.16	Integration d'un agent IA	14
6.17	Auto-remediation	14
6.18	Generation de rapports	15
7	Conclusion	16
8	Annexe — Agents IA et agents automatiques du projet	17
8.1	Agent IA de decision NOC	17
8.2	Agent AIOps d'auto-remediation	18

8.3	Agent collecteur d'infrastructure	20
8.4	Moteur d'alertes intelligent	21
8.5	Resume du fonctionnement global	22
8.6	Pipeline de traitement de l'agent IA	22

Table des figures

2.1	Architecture A : Pipeline d'alerting lineaire	4
2.2	Architecture B : ajout d'une boucle de fermeture automatique (auto-remediation) et d'un audit trail	5
2.3	Architecture C : separation du flux de stockage/visualisation et du flux d'alerting	5
3.1	Architecture D (Cible) : integration d'un AI Agent, d'une passerelle WebSocket pour le temps reel et d'une couche d'abstraction Mock/Production	7
4.1	Schéma logique du pipeline de traitement unifié de l'Agent IA	10
8.1	Flux global : Collector – Moteur d'alertes – Agent IA NOC – Agent AIOps – API/Dashboard	17

1. Contexte du projet

Ce document est realise dans le cadre du stage effectue chez CIRES Technologies. Il porte sur la conception d'une plateforme de Cloud Monitoring Overview integrant un Agent IA, destinee a superviser une infrastructure Cloud (Kubernetes, OpenShift, VMware) et a assister les administrateurs dans la prise de decision.

La plateforme cible s'inspire des standards du marche (Datadog, Grafana Cloud, Dynatrace, New Relic, Azure Monitor, Elastic, Cisco ThousandEyes) et doit, dans une premiere phase, fonctionner integralement en mode Mock : toutes les donnees (metriques, logs, traces, alertes, VMs, pods, clusters) sont generees automatiquement et evoluent en temps reel, avant d'etre connectees aux API reelles (Grafana, Prometheus, Loki, Tempo, OpenTelemetry, Kubernetes, OpenShift, VMware).

Avant de lancer le developpement, il est necessaire de comparer les patterns d'architecture de pipeline de monitoring envisageables, afin de choisir une base solide, evolutive et alignee avec les exigences du cahier des charges (Clean Architecture, Repository Pattern, Service Layer, interchangeabilite Mock/Production via une variable d'environnement unique).

1.1 Objectif du benchmark

L'objectif est de comparer trois patterns d'architecture de pipeline de monitoring couramment rencontres dans l'industrie, d'en degager les forces et les limites, puis de proposer une architecture cible adaptee au projet Cloud AI Monitor de CIRES Technologies.

- Identifier le pattern le plus adapte a une plateforme de supervision augmentee par IA.
- Determiner comment integrer un moteur de decision intelligent (AI Agent) sans complexifier inutilement le pipeline.
- Garantir une architecture decouplee, ou le mode Mock et le mode Production (APIs reelles) sont interchangeables.
- Poser les bases d'une architecture scalable, auditable et maintenable en environnement d'entreprise.

2. Les trois patterns d'architecture etudes

Trois architectures de pipeline de monitoring ont ete identifiees et modelisees a partir des schemas de reference du projet. Chacune represente un niveau de maturite different, allant de l'alerting basique a l'observabilite complete.

2.1 Architecture A — Pipeline d'alerting lineaire

Ce pattern represente le modele le plus simple : les donnees issues des sources sont collectees, evaluees par un moteur de regles a seuils statiques, puis notifiees a un operateur qui intervient manuellement. Il ne comporte ni remediation automatique, ni couche d'observabilite dediee.

Flux : Sources → Collector → Rules Engine → Notifier → Ops

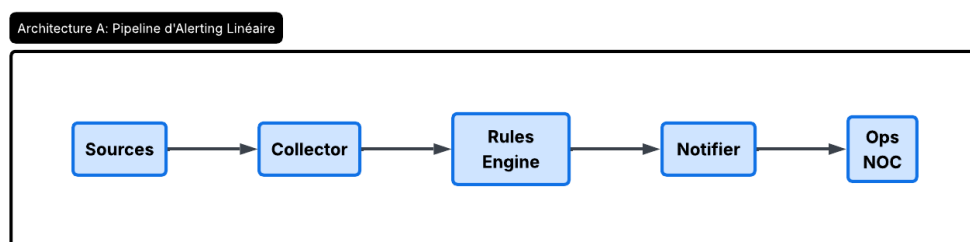


FIGURE 2.1 – Architecture A : Pipeline d'alerting lineaire

Forces	Limites
Simplicité de mise en oeuvre et de maintenance	Aucune automatisisation de la remediation
Cout d'infrastructure minimal	Pas de visualisation ni d'exploration des donnees historiques
Ideal pour un MVP ou une preuve de concept	MTTR eleve car toute action reste manuelle
	Ne passe pas a l'echelle sur une infrastructure multi-cloud

TABLE 2.1 – Forces et limites de l'Architecture A

2.2 Architecture B — Pipeline avec agent de remediation

Ce pattern enrichit le pipeline de base avec un Remediation Agent capable de declencher des actions correctives automatiques via des actuators (Kubernetes, VM, scripts), ainsi qu'un flux

de notification et d'audit relie a un dashboard de suivi.

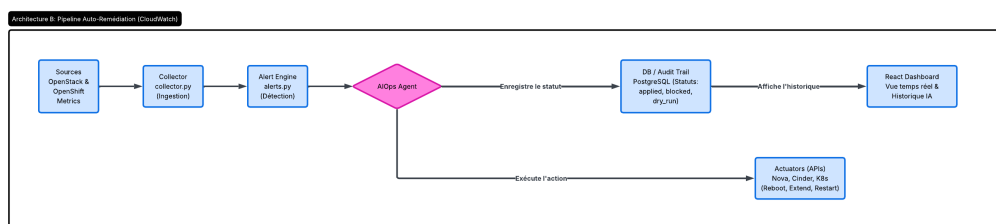


FIGURE 2.2 – Architecture B : ajout d'une boucle de fermeture automatique (auto-remediation) et d'un audit trail

Forces	Limites
Reduction forte du MTTR grace a l'automatisation	Risque operationnel si les actions automatiques sont mal calibrees
Tracabilite complete via l'audit trail	Necessite une gouvernance stricte (validation, rollback)
Le Remediation Agent prefigure l'AI Agent du projet CIRES	Absence de couche d'observabilite exploratoire (pas de type Grafana)

TABLE 2.2 – Forces et limites de l'Architecture B

2.3 Architecture C — Pipeline d'observabilite (type Grafana Stack)

Ce pattern separe explicitement le flux d'observabilite (stockage en base de series temporelles puis visualisation via un frontend type Grafana) du flux d'alerting (moteur de regles puis notification). C'est le modele le plus proche des stacks Prometheus / Loki / Tempo / Grafana mentionnees dans le cahier des charges.

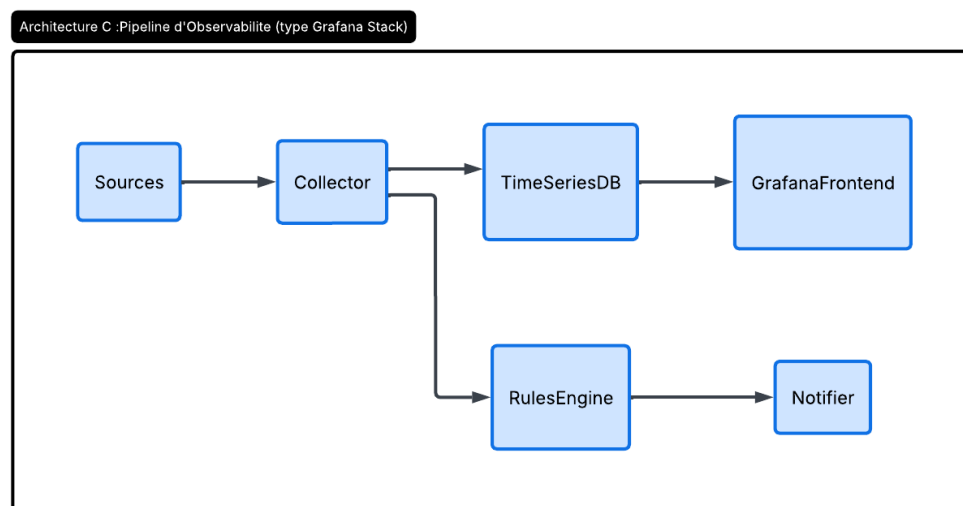


FIGURE 2.3 – Architecture C : separation du flux de stockage/visualisation et du flux d'alerting

Forces	Limites
Excellente visibilité et exploration des données historiques	Aucune automatisation ni intelligence de décision intégrée
Architecture directement compatible avec Grafana, Prometheus, Loki, Tempo	Coût d’infrastructure plus élevé (stockage de séries temporelles)
Scalabilité élevée, standard de l’industrie	Nécessite une couche supplémentaire pour l’IA et la remédiation

TABLE 2.3 – Forces et limites de l’Architecture C

3. Architecture cible recommandee pour Cloud AI Monitor

3.1 Architecture D — Pipeline hybride (Cible)

Ce pattern est une approche sur mesure qui fusionne l'observabilite de l'Architecture C (collecte centralisee, stockage, exploration des donnees historiques) avec la boucle d'analyse intelligente de l'Architecture B. Plutot que de s'appuyer sur un simple agent de remediation automatise, cette architecture integre un AI Agent au cœur du pipeline. Ce moteur analyse de maniere croisee les metriques, logs et traces, detecte les anomalies, calcule un Health Score global et propose des recommandations predictives. C'est l'architecture hybride officiellement adoptee pour le projet Cloud AI Monitor.

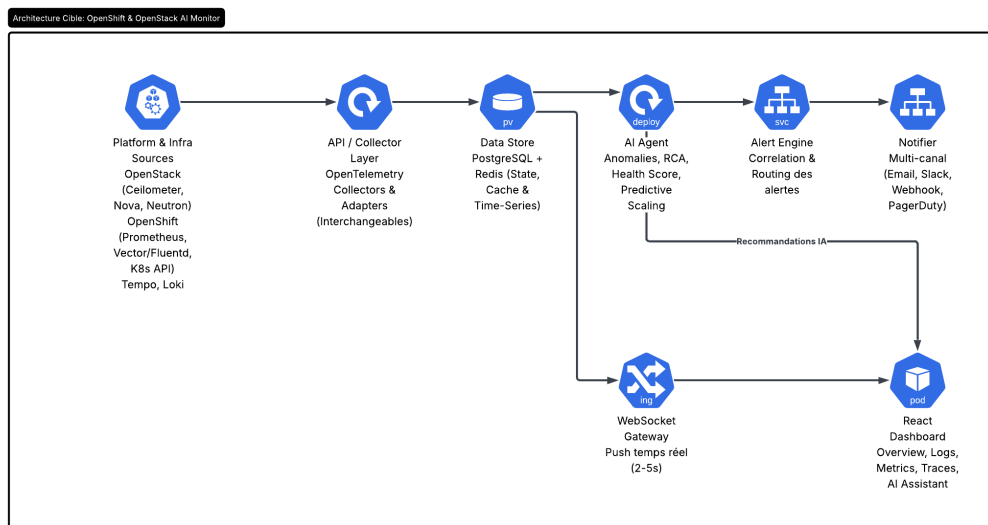


FIGURE 3.1 – Architecture D (Cible) : integration d'un AI Agent, d'une passerelle WebSocket pour le temps reel et d'une couche d'abstraction Mock/Production

Forces	Limites
L’AI Agent est au centre du flux, agissant comme un copilote pour reduire le MTTR via des recommandations contextuelles.	Necessite la mise en place d’une Clean Architecture stricte pour gerer l’abstraction des services.
L’interface commune (Service Layer) permet de basculer instantanement des donnees generees vers les API reelles via une simple variable d’environnement.	Exige une infrastructure supplementaire (Redis pour le cache, WebSockets) pour assurer la diffusion des donnees en temps reel.
L’IA assiste la prise de decision (Alert Engine) sans imposer d’actions correctives aveugles (remediation avec validation humaine).	La normalisation des donnees issues de multiples sources (Prometheus, Loki, Tempo, etc.) demande une API robuste (FastAPI).

TABLE 3.1 – Forces et limites de l’Architecture D (Cible)

3.2 Principes cles de l’architecture cible

- **Couche Sources / Mock interchangeable** : chaque service (GrafanaService, PrometheusService, LokiService, TempoService, KubernetesService, OpenShiftService, VMwareService) expose une interface commune, implementee soit par un Mock Service, soit par un client reel, bascule via une seule variable d’environnement.
- **API / Collector Layer** : couche FastAPI en Clean Architecture (Repository Pattern, Service Layer, Dependency Injection) qui normalise les donnees quelle que soit leur origine.
- **Data Store** : PostgreSQL pour la persistance structuree (alertes, ressources, historique) et Redis pour le cache et l’etat temps reel.
- **AI Agent** : moteur d’analyse (simule au depart, remplaçable par un vrai LLM) qui alimente a la fois l’Alert Engine et le Dashboard via des cartes de recommandation, sans se substituer a la decision humaine.
- **Alert Engine** : correlation, deduplication et reduction du bruit avant notification, avec cycle de vie complet de l’alerte (Acknowledged, Resolved, Assigned, Historique).
- **WebSocket Gateway** : diffusion temps reel (toutes les 2 a 5 secondes) vers le Dashboard React pour un rendu vivant, conforme au mode Mock demande.

3.3 Adequation avec le cahier des charges

Cette architecture cible respecte les contraintes explicitement demandees : aucune logique metier ne depend du mode Mock, le Mock remplace uniquement les APIs ; l’AI Agent est conçu des le depart pour etre remplace par un vrai modele de langage sans refonte structurelle ; et l’ensemble des modules fonctionnels (Dashboard, Infrastructure, Logs, Metrics, Traces, Alerts, AI Agent, Reports, Settings, Authentication) s’appuient sur le meme pipeline de donnees unifie.

4. Architecture Cible

4.1 Pipeline de traitement de l'Agent IA

Le moteur évalue les données entrantes via un pipeline unifié, conçu pour traiter les flux de télémétrie en temps réel et déclencher des actions de remédiation autonomes. Ce pipeline se décompose en trois couches séquentielles et interconnectées :

Couche d'Ingestion (Ingestion Layer)

Agrège les métriques simulées (consommation CPU/RAM, latence réseau), les événements de logs classifiés par sévérité (INFO, WARN, CRITICAL), et les spans de traçage distribué. Cette couche agit comme un buffer de haute performance, normalisant les données hétérogènes pour le modèle sous-jacent.

Modèle de Raisonnement (Reasoning Model)

À partir des données normalisées, le modèle calcule un vecteur d'anomalie multidimensionnel. Ce vecteur est noté $\vec{A} = [m, l, t]$, où m représente le score d'anomalie des métriques de performance, l l'intensité et la criticité des logs, et t l'impact des goulots d'étranglement issus des traces réseau.

Moteur de Décision (Decision Engine)

Le moteur évalue dynamiquement l'état du système. Si le score de santé (Health Score) calculé descend en dessous du seuil de tolérance défini, il confronte le vecteur d'anomalie $\vec{A}$ à une base de connaissances interne (Knowledge Base) afin de produire et d'exécuter une proposition de remédiation actionnable.

Modélisation du Score de Santé (Health Score)

Le Moteur de Décision s'appuie sur une évaluation quantitative pour déclencher ses actions. Le score de santé globale, noté H , peut être formalisé par l'équation suivante :

$$H = 100 - (\alpha \cdot m + \beta \cdot l + \gamma \cdot t)$$

Où α , β , et γ représentent les poids de pondération attribués à chaque dimension de la télémétrie, ajustables selon la criticité de l'infrastructure supervisée. Dès que $H < S_{seuil}$, le Moteur de Décision initie le processus de remédiation.

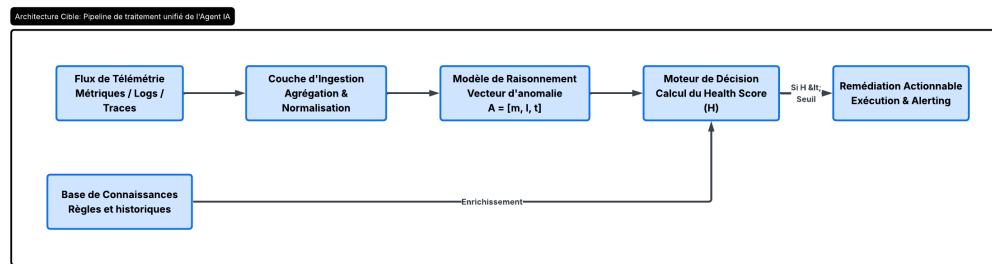


FIGURE 4.1 – Schéma logique du pipeline de traitement unifié de l'Agent IA

5. Tableau comparatif

Le tableau ci-dessous synthetise la comparaison des trois architectures etudiees selon des criteres techniques et operationnels, et introduit l'architecture cible (D) proposee pour le projet Cloud AI Monitor.

Critere	A. Alerting lineaire	B. Remediation	C. Observabilite	D. Cible CIRES (hybride)
Complexite de mise en oeuvre	Faible	Elevee	Moyenne	Moyenne (par etapes)
MTTR (temps moyen de resolution)	Eleve (100% manuel)	Tres faible (auto-remediation)	Moyen (analyse visuelle)	Faible (IA assiste la decision)
Niveau d'automatisation	Aucun	Fort (actuators)	Aucun	Progressif (recos → actions)
Visibilite / exploration des donnees	Faible	Moyenne	Elevee (type Grafana)	Elevee (dashboard + AI Assistant)
Risque operationnel	Faible	Moyen a eleve	Faible	Maitrise (validation humaine)
Scalabilite	Faible	Moyenne	Elevee	Elevee (microservices, cache Redis)
Auditabilite / tracabilite	Faible	Elevee	Moyenne	Elevee (AuditLogs dedies)
Adequation avec un AI Agent	Faible	Elevee (agent = moteur de decision)	Moyenne (necessite couche IA)	Native (AI Agent au coeur du flux)
Cout d'infrastructure	Faible	Moyen	Eleve (stack TSDB + Grafana)	Progressif (mock → APIs reelles)
Cas d'usage type	PME / MVP	SRE avance / Cloud Native	Observabilite entreprise	Plateforme IA de supervision Cloud

TABLE 5.1 – Comparatif des architectures A, B, C et D

Il ressort de cette comparaison qu'aucune des trois architectures prises isolement ne repond pleinement aux exigences du cahier des charges CIRES Technologies : l'architecture A manque d'automatisation et de visibilite, l'architecture B manque d'observabilite exploratoire, et l'architecture C manque d'intelligence de decision. Le projet necessite une architecture hybride.

6. Taches et fonctionnalites

6.1 Authentification securisee

La plateforme doit permettre a chaque utilisateur autorise de s'authentifier avant d'accéder aux fonctionnalites. L'authentification repose sur des identifiants verifiés et sur la generation d'un jeton d'accès. Objectifs : proteger les donnees de supervision, identifier l'utilisateur, appliquer des droits selon le role et tracer les actions sensibles.

6.2 Tableau de bord intelligent

Le tableau de bord fournit une synthese de l'infrastructure. Il presente des indicateurs tels que le nombre de machines virtuelles, de pods, d'alertes actives, d'alertes critiques et le score global de sante. Il permet aux responsables et operateurs de comprendre rapidement l'etat du systeme.

6.3 Supervision des machines virtuelles

La plateforme supervise les machines virtuelles de l'environnement cloud. Elle affiche :

- Identifiant logique.
- Nom.
- Etat.
- Flavor ou capacite.
- Hote.
- Consommation CPU.
- Consommation RAM.
- Date de derniere mise a jour.

6.4 Supervision des pods

La supervision des pods permet de suivre les workloads conteneurises. Les informations principales sont :

- Namespace.
- Nom du pod.
- Etat.
- Node d'execution.
- Image.
- Nombre de redemarrages.
- Consommation CPU.
- Consommation memoire.

6.5 Surveillance des metriques

Les metriques permettent d’observer l’evolution des ressources. Elles sont utilisees pour detecter une saturation, analyser une tendance, comprendre l’impact d’un incident, alimenter les alertes et produire des rapports.

6.6 Collecte automatique des donnees

La collecte automatique evite les interventions manuelles. Elle interroge regulierement les sources cloud afin de maintenir une vision a jour de l’infrastructure. La collecte peut fonctionner avec des donnees reelles provenant de l’infrastructure ou des donnees simulees pour les tests et demonstrations.

6.7 Detection d’anomalies

La detection d’anomalies repose sur des regles et des seuils. Exemples :

- CPU superieur a un seuil critique.
- RAM superieure a un seuil critique.
- VM en etat d’erreur.
- Pod en echec.
- Nombre eleve de redemarrages.

6.8 Gestion des alertes

La gestion des alertes permet la consultation des alertes actives, le filtrage par statut ou severite, l’acquittement par un operateur, la resolution manuelle ou automatique, l’affichage de la recommandation IA et l’affichage du statut de remediation.

6.9 Historique

L’historique conserve les donnees de metriques et d’alertes. Il permet une analyse retrospective et facilite la comprehension des incidents recurrents.

6.10 Journalisation

La journalisation permet de conserver les traces des evenements importants :

- Creation d’alerte.
- Resolution.
- Acquittement.
- Action de remediation.
- Erreurs de collecte.
- Operations sensibles.

6.11 Recherche et filtrage

La recherche et le filtrage ameliorent l’exploitation du systeme. Les utilisateurs peuvent isoler les alertes critiques, les ressources d’un namespace, les VMs problematiques ou les elements recents.

6.12 Gestion des utilisateurs

La gestion des utilisateurs permet d’administrer les comptes ayant acces a la plateforme. Elle inclut l’identification, le role et les informations necessaires au controle d’accès.

6.13 Gestion des roles

6.14 Parametrage du systeme

Le parametrage permet de definir :

- Seuils CPU et RAM.
- Frequence de collecte.
- Mode simulation ou mode reel.
- Activation de l’auto-remediation.
- Seuil minimum de score IA.
- Limites de securite.
- Connecteurs cloud.

6.15 Visualisation des performances

Les performances sont visualisees sous forme de courbes, cartes, compteurs et tableaux. Les indicateurs aident a identifier rapidement les ressources sous pression.

6.16 Integration d’un agent IA

L’agent IA analyse les alertes en fonction de leur severite, de leur contexte, du depassement de seuil et de la recurrence. Il produit :

- Un score.
- Une categorie.
- Une raison.
- Une recommandation.
- Un niveau de confiance.

6.17 Auto-remediation

L’auto-remediation consiste a preparer ou executer une action corrective. Elle reste controlee par des garde-fous :

- Mode simulation.
- Score minimum.
- Limite du nombre d’actions.
- Limitation des namespaces.
- Validation humaine pour actions sensibles.
- Plafonds de capacite.

6.18 Generation de rapports

La generation de rapports permet de fournir une synthese de l’activite, de l’etat des ressources, des incidents et des indicateurs de disponibilite.

7. Conclusion

La comparaison des trois patterns d'architecture met en evidence des compromis clairs entre simplicité, automatisation et observabilité. Pour une plateforme d'entreprise de niveau Datadog / Grafana Cloud intégrant un Agent IA, aucun pattern seul ne suffit : l'architecture retenue pour Cloud AI Monitor combine la robustesse d'une stack d'observabilité (architecture C) avec la capacité de décision automatisée d'un agent (architecture B), le tout organise autour d'un AI Agent central capable d'évoluer d'un moteur simulateur vers un véritable modèle de langage.

Cette architecture hybride constitue la base technique recommandée pour la suite du développement du projet de stage chez CIRES Technologies, en garantissant scalabilité, auditabilité et une transition progressive et maîtrisée du mode Mock vers les APIs de production (Grafana, Prometheus, Loki, Tempo, Kubernetes, OpenShift, VMware).

8. Annexe — Agents IA et agents automatiques du projet

Le projet contient deux agents IA principaux et deux composants automatiques qui alimentent ces agents. Le flux global est le suivant :

Collector → **Moteur d’alertes** → **Agent IA de decision NOC** → **Agent AIOps d’auto-remediation** → **API + Dashboard**

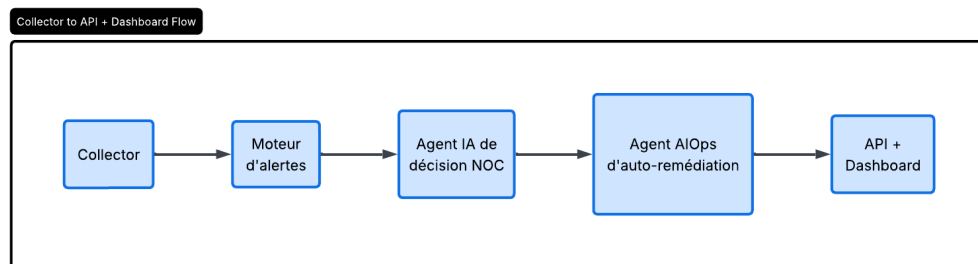


FIGURE 8.1 – Flux global : Collector – Moteur d’alertes – Agent IA NOC – Agent AIOps – API/Dashboard

8.1 Agent IA de decision NOC

Fichier : `backend/ai_agent.py`

Role

- Analyser chaque alerte creee par le moteur d’alertes.
- Calculer un score d’impact entre 0 et 100.
- Classer l’alerte dans une categorie comme *compute*, *container* ou *infrastructure*.
- Donner une decision operationnelle : *escalate*, *investigate*, *watch* ou *suppress*.
- Produire une recommandation lisible pour l’operateur NOC.
- Enregistrer la confiance de l’analyse.

Fonctionnement detaille

La fonction `analyze_alert(db, alert)` recoit une alerte et procede ainsi :

- Elle demarre avec un score de base de 20.
- Elle augmente le score selon la severite :
 - *critical* ajoute 45 points;

- *warning* ajoute 25 points ;
- *info* ajoute 8 points.
- Elle compare la valeur mesuree au seuil :
 - si la valeur depasse fortement le seuil, le score augmente davantage ;
 - si elle depasse simplement le seuil, le score augmente moins.
- Elle regarde le nom de la regle :
 - *vm.** classe l’alerte en *compute* ;
 - *pod.** classe l’alerte en *container* ;
 - une regle contenant *status* ajoute un risque d’etat anormal ;
 - une regle contenant *restarts* ajoute un risque de redemarrages repetes.
- Elle verifie la ressource liee :
 - si la VM est en *ERROR*, le score augmente ;
 - si le pod est *Failed* ou *Unknown*, le score augmente.
- Elle detecte les alertes repetees sur la derniere heure.
- Elle reduit legerement le score si l’alerte est deja acquittee.
- Elle limite le score final entre 0 et 100.
- Elle transforme le score en decision :
 - ≥ 80 : *escalate* ;
 - ≥ 55 : *investigate* ;
 - ≥ 35 : *watch* ;
 - < 35 : *suppress*.

La fonction `apply_decision(db, alert)` sauvegarde le resultat dans l’alerte : `ai_score`, `ai_decision`, `ai_category`, `ai_reason`, `ai_recommendation`, `ai_confidence`, `ai_updated_at`.

Exemple

Alerte : RAM critique sur VM app-01

Severite : critical

Valeur : 94%

Seuil : 90%

Resultat agent NOC :

- score = 80+
- decision = escalate
- categorie = compute
- recommandation = intervention rapide controlee

8.2 Agent AIOps d’auto-remediation

Fichier : `backend/remediation_agent.py`

Role

- Choisir automatiquement une action corrective.
- Construire un plan d’intervention explicable.
- Verifier les garde-fous avant execution.
- Simuler l’action en mode dry-run.
- Appliquer l’action reelle seulement si la configuration l’autorise.
- Garder une trace dans l’alerte : action, statut, message et date.

Fonctionnement detaille

`execute_remediation(db, alert, operator, force)` est le point d’entree principal. L’agent appelle d’abord `apply_decision()` pour recalculer l’analyse IA NOC.

`choose_action(alert)` choisit l’action selon le texte, la regle et la ressource :

- disque, stockage, storage, espace → `extend_storage`;
- RAM, memory, memoire → `scale_memory`;
- CPU → `scale_compute`;
- VM bloquee ou statut VM anormal → `recover_vm`;
- pod en redemarrages repetes → `restart_workload`;
- pod avec statut anormal → `isolate_workload`;
- cas inconnu → `open_incident`.

`build_plan(db, alert)` cree un plan avec :

- l’action cible;
- la ressource concerne;
- le niveau de risque : low, medium, high;
- la raison;
- les etapes d’execution;
- l’impact estime;
- si une validation humaine est requise.

`_guardrails_allow(db, action)` bloque les boucles dangereuses : l’agent compte les actions similaires executees sur la derniere heure ; si le nombre depasse `REMEDATION_MAX_ACTIONS_PER_HOUR`, l’action est bloquee.

L’action est autorisee seulement si :

- l’alerte est active;
- les garde-fous sont OK;
- `AUTO_REMEDIATION_ENABLED=true`;
- le score IA est superieur ou egal a `AUTO_REMEDIATION_MIN_SCORE`;
- l’action ne demande pas de validation humaine;
- ou bien `force=true` est envoye par l’operateur.

Si `AUTO_REMEDIATION_DRY_RUN=true`, l’agent ne modifie pas l’infrastructure : il ecrit seulement le plan dans l’alerte avec le statut `dry_run`. Si `AUTO_REMEDIATION_DRY_RUN=false`, `_run_real_action()` execute l’action reelle.

Actions reelles supportees

Action	Cas traite	Execution
extend_storage	Stockage sature	Trouve le premier volume attache, calcule la nouvelle taille, verifie REMEDIATION_MAX_STORAGE_GB, appelle Cinder extend_volume
scale_memory	RAM saturee	Lit le flavor actuel, cherche le flavor cible dans REMEDIATION_MEMORY_FLAVOR_MAP, lance un resize Nova
recover_vm	VM bloquee ou en erreur	Applique hard_reboot, stop ou live_migrate selon REMEDIATION_VM_RECOVERY_ACTION
restart_workload	Pod instable	Verifie que le namespace est autorise, supprime le pod pour que Kubernetes le recree
quarantine_vm	Incident critique	Stoppe la VM via OpenStack
open_incident	Cas non automatise	Recommande une intervention humaine

TABLE 8.1 – Actions reelles supportees par l’agent AIOps

Statuts possibles

Statut	Signification
recommended	Action conseilllee mais non executee
dry_run	Simulation executee, aucun changement reel
applied_mock	Action simulee en mode mock
applied	Action reelle executee
blocked	Action refusee par manque de donnees, configuration ou garde-fou

TABLE 8.2 – Statuts possibles d’une action de remediation

Exemple — Stockage

Alerte : stockage critique sur VM app-01

Action choisie : extend_storage

Plan :

1. Verifier que l’alerte est active.
2. Identifier le volume attache.
3. Ajouter REMEDIATION_STORAGE_INCREMENT_GB.
4. Verifier la limite REMEDIATION_MAX_STORAGE_GB.
5. Executer l’extension Cinder.

8.3 Agent collecteur d’infrastructure

Fichier : backend/collector.py

Ce composant n’est pas un agent IA, mais il alimente les agents IA avec les donnees d’infrastructure.

Role

- Collecter les VMs OpenStack.
- Collecter les pods OpenShift/Kubernetes.
- Recupérer les metriques CPU, RAM, disque et redemarrages.
- Enregistrer les ressources et metriques en base.
- Declencher l’évaluation des alertes.

Fonctionnement

- `start_scheduler()` lance APScheduler.
- Toutes les `COLLECT_INTERVAL_SECONDS`, la fonction `collect_all()` démarre.
- `collect_openstack(db)` recupere les VMs :
 - en mode mock : utilise `mock_data.get_mock_vms()` ;
 - en mode reel : appelle OpenStack via `openstack_client.list_servers()`.
 Pour chaque VM, le collecteur met a jour la table `virtual_machines`, ajoute une ligne dans `vm_metrics` et appelle `alert_engine.evaluate_vm()`.
- `collect_openshift(db)` recupere les pods :
 - en mode mock : utilise `mock_data.get_mock_pods()` ;
 - en mode reel : appelle Kubernetes/OpenShift.
 Pour chaque pod, le collecteur met a jour la table `pods`, ajoute une ligne dans `pod_metrics` et appelle `alert_engine.evaluate_pod()`.

8.4 Moteur d’alertes intelligent

Fichier : `backend/alerts.py`

Ce composant n’est pas un LLM, mais il connecte la supervision avec les agents IA.

Role

- Transformer les metriques en alertes.
- Eviter les doublons d’alertes actives.
- Resoudre automatiquement les alertes quand la metrique revient a la normale.
- Appeler l’agent IA NOC.
- Appeler l’agent AIOps si necessaire.
- Envoyer une notification email si configure.

Fonctionnement

- `evaluate_vm(db, vm, cpu, ram)` verifie les seuils VM : CPU warning, CPU critical, RAM warning, RAM critical, VM en statut *ERROR*.
- `evaluate_pod(db, pod, restart_count)` verifie les seuils pods : pod *Failed* ou *Unknown*, redemarrages superieurs ou egaux a 5.
- Quand une condition est vraie, `_create_alert()` cree une alerte active. Avant creation, `_find_active()` verifie qu’une alerte identique n’existe pas deja.
- Apres creation, le moteur appelle :
 - `apply_decision(db, alert)` pour l’analyse IA ;
 - `auto_remediate_if_needed(db, alert)` pour l’auto-remediation ;
 - `send_alert_email(alert)` pour notifier.
- Quand la condition n’est plus vraie, `_resolve_alert()` passe l’alerte en *resolved*.

8.5 Resume du fonctionnement global

1. Le collecteur lit OpenStack et OpenShift.
2. Les metriques sont stockees en base.
3. Le moteur d’alertes detecte CPU/RAM/VM/pod anormal.
4. L’agent IA NOC calcule le score et la decision.
5. L’agent AIOps choisit une action corrective.
6. Les garde-fous verifient le risque.
7. En dry-run, l’agent simule.
8. En mode reel, il execute via OpenStack ou Kubernetes.
9. Le dashboard affiche le score IA, la recommandation et le statut de remediation.

8.6 Pipeline de traitement de l’agent IA

Le moteur évalue les données entrantes à travers un pipeline unifié :

- **Couche d’ingestion** : agrège les métriques simulées (CPU/RAM), les événements de logs (INFO, WARN, CRITICAL) ainsi que les spans de tracing.
- **Modèle de raisonnement** : calcule un vecteur d’anomalie $\vec{A} = [m, l, t]$ représentant respectivement les métriques, les logs et les traces.
- **Moteur de décision** : si le *Health Score* calculé descend en dessous du seuil défini, le vecteur d’anomalie est comparé à une base de connaissances interne afin de produire une proposition de remédiation actionnable.